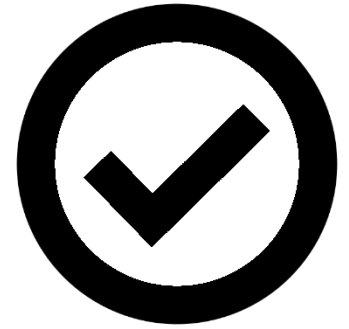




**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential



C++ PROGRAMMING

(335)

STATE 2021

APPLICATION KNOWLEDGE:

Caesar Cipher

_____ (400 points)

TOTAL POINTS

_____ (400 points)

Test Time: 90 minutes

Caesar Cipher

One of the simplest methods of encryption is the Caesar cipher, also known as a shift cipher. It works on a simple premise – when each letter is shifted by a fixed amount, the resulting word is impossible to read unless that amount is known. In this exam, you will create a C++ console application that accepts a string input and an amount to shift, then shows the encrypted string. To show that it isn't random, the encrypted string is then shown decrypted, and it should match what was input.

To simplify the encryption, the shift should be limited to a value between 1 and 5, and to prevent issues with this shift input validation should be looped until a value is properly entered.

Additionally, a Cipher class should be created to facilitate the encryption and decryption of input using the Caesar shift. Case-sensitivity should also be preserved during encryption and decryption. Note that when decrypting a value encrypted using a Caesar cipher, the process of decryption is identical to encryption but with a reversed shift.

```
Enter a word to encrypt (or EXIT to close application): cat
Enter an amount to shift the input by (must be between 1 and 5): -1
Enter an amount to shift the input by (must be between 1 and 5): 0
Enter an amount to shift the input by (must be between 1 and 5): 6
Enter an amount to shift the input by (must be between 1 and 5): 3

The encrypted value is: fdw
When decrypted again the result is: cat

Enter a word to encrypt (or EXIT to close application): dog
Enter an amount to shift the input by (must be between 1 and 5): 2

The encrypted value is: fqi
When decrypted again the result is: dog
```

Figure 1: Sample output for input cat and dog with shifts of 3 and 2, showing sample input validation.

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
64	01000000	100	40	@	96	01100000	140	60	`
65	01000001	101	41	A	97	01100001	141	61	a
66	01000010	102	42	B	98	01100010	142	62	b
67	01000011	103	43	C	99	01100011	143	63	c
68	01000100	104	44	D	100	01100100	144	64	d
69	01000101	105	45	E	101	01100101	145	65	e
70	01000110	106	46	F	102	01100110	146	66	f
71	01000111	107	47	G	103	01100111	147	67	g
72	01001000	110	48	H	104	01101000	150	68	h
73	01001001	111	49	I	105	01101001	151	69	i
74	01001010	112	4A	J	106	01101010	152	6A	j
75	01001011	113	4B	K	107	01101011	153	6B	k
76	01001100	114	4C	L	108	01101100	154	6C	l
77	01001101	115	4D	M	109	01101101	155	6D	m
78	01001110	116	4E	N	110	01101110	156	6E	n
79	01001111	117	4F	O	111	01101111	157	6F	o
80	01010000	120	50	P	112	01110000	160	70	p
81	01010001	121	51	Q	113	01110001	161	71	q
82	01010010	122	52	R	114	01110010	162	72	r
83	01010011	123	53	S	115	01110011	163	73	s
84	01010100	124	54	T	116	01110100	164	74	t
85	01010101	125	55	U	117	01110101	165	75	u
86	01010110	126	56	V	118	01110110	166	76	v
87	01010111	127	57	W	119	01110111	167	77	w
88	01011000	130	58	X	120	01111000	170	78	x
89	01011001	131	59	Y	121	01111001	171	79	y
90	01011010	132	5A	Z	122	01111010	172	7A	z

Figure 2: An Ascii table that can be used for performing the necessary shift.

Requirements:

1. You must create a C++ console application named CPP_335_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CPP_335_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. A Cipher class shall be implemented to facilitate the Caesar shift operations and must support both encryption and decryption operations.
4. If the target word to encrypt is EXIT, the application shall stop looping and instead prompt for the user to press a key to exit the application.
5. Otherwise, the application shall continuously prompt for a word to encrypt, a number to shift by, and then report the encrypted and then decrypted word, until EXIT is entered.
6. When entering the shift amount, the application shall continuously ask for additional input until amount is between 1 and 5.

Your application will be scored on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 pts
The project is named according to the naming conventions _____ 10 pts

Program Execution

Code copied to flash drive and the program runs from the flash drive _____ 10 pts

If the program does not execute, then the remaining items in this section receive a score of zero.

Application prompts user for a word to encrypt _____ 20 pts
Application prompts user for an amount to shift by _____ 20 pts
Application performs proper validation for amount to shift by _____ 35 pts
Application performs proper Caesar cipher according to specification _____ 50 pts
Caesar cipher properly preserves case sensitivity _____ 50 pts
Application repeats until EXIT is entered as word to encrypt _____ 20 pts
Application pauses prior to exiting _____ 20 pts

Source Code Review

Code is commented at the top, for each function, and as needed _____ 20 pts
Code uses reasonable and consistent variable naming conventions _____ 20 pts
Cipher class exists and has suitable functions and constructors _____ 50 pts
Cipher functions are reasonably modular in design _____ 35 pts
Input validation is performed properly _____ 30 pts

Total Points: _____ 400 pts

Sample Output for Verification

```
Enter a word to encrypt (or EXIT to close application): BUSINess
Enter an amount to shift the input by (must be between 1 and 5): 0
Enter an amount to shift the input by (must be between 1 and 5): 6
Enter an amount to shift the input by (must be between 1 and 5): 5

The encrypted value is: GZXNSjxx
When decrypted again the result is: BUSINess

Enter a word to encrypt (or EXIT to close application): surprising
Enter an amount to shift the input by (must be between 1 and 5): 2

The encrypted value is: uwtrtkukpi
When decrypted again the result is: surprising

Enter a word to encrypt (or EXIT to close application): elephant
Enter an amount to shift the input by (must be between 1 and 5): 5

The encrypted value is: jqjumfsy
When decrypted again the result is: elephant

Enter a word to encrypt (or EXIT to close application): EXIT
Press any key to continue . . .
```

Completed Source Code

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

class Cipher
{
private:
    string input;

public:
    Cipher()
    {

    }

    Cipher(string newInput)
    {
        SetInput(newInput);
    }

    void SetInput(string newInput)
    {
        input = newInput;
    }

    string Encrypt(int shift)
```

```

    {
        char ch;
        string result = "";

        for (int i = 0; i < input.length(); i++)
        {
            ch = input[i];
            if (isalpha(ch))
            {
                if (isupper(ch))
                {
                    ch += shift;
                    if (ch > 'Z')
                        ch += 'A' - 1 - 'Z';
                    result += char(ch);
                }
                else
                {
                    ch += shift;
                    if (ch > 'z')
                        ch += 'a' - 1 - 'z';
                    result += char(ch);
                }
            }
        }

        return result;
    }

string Decrypt(int shift)
{
    char ch;
    string result = "";

    for (int i = 0; i < input.length(); i++)
    {
        ch = input[i];
        if (isalpha(ch))
        {
            if (isupper(ch))
            {
                ch -= shift;
                if (ch < 'A')
                    ch -= 'A' - 1 - 'Z';
                result += char(ch);
            }
            else
            {
                ch -= shift;
                if (ch < 'a')
                    ch -= 'a' - 1 - 'z';
                result += char(ch);
            }
        }
    }

    return result;
}

};

```

```
int main()
{
    string input;
    string encryptedVal;
    string decryptedVal;
    bool running = true;
    int shiftCnt = 0;

    while (running)
    {
        cout << "Enter a word to encrypt (or EXIT to close application): ";
        getline(cin, input);

        if (!input.compare("EXIT"))
        {
            running = false;
        }
        else
        {
            Cipher enc(input);
            Cipher dec;

            do
            {
                cout << "Enter an amount to shift the input by (must be
between 1 and 5): ";
                getline(cin, input);
                shiftCnt = atoi(input.c_str());
            } while (shiftCnt > 5 || shiftCnt < 1);

            encryptedVal = enc.Encrypt(shiftCnt);
            dec.SetInput(encryptedVal);
            decryptedVal = dec.Decrypt(shiftCnt);

            cout << "\nThe encrypted value is: " << encryptedVal << endl;
            cout << "When decrypted again the result is: " << decryptedVal <<
endl << endl;
        }
    }

    system("pause");
    return 0;
}
```